

Exercises — Dynamic Programming from the Book

Exercise 4.1 — Action Values

If π is the **equiprobable random policy**, determine:

1. $q_\pi(11, \text{down})$
2. $q_\pi(7, \text{down})$

Use the definition of the action-value function:

$$q_\pi(s, a) = E_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a]$$

Exercise 4.2 — Adding a New State

Suppose a new state **15** is added to the Gridworld below state 13.

The actions from state 15 lead to:

Action Next State

Left	12
Up	13
Right	14
Down	15

Assume that transitions from the original states remain unchanged.

Questions:

1. What is $v_\pi(15)$ under the equiprobable random policy?
 2. Now suppose action **Down** from state 13 also leads to the new state 15. What is $v_\pi(15)$ under the equiprobable random policy?
-

Exercise 4.3 — Action-Value Function

Derive equations analogous to Equations (4.3), (4.4), and (4.5) for the action-value function q_π .

Specifically, derive:

1. The Bellman equation for $q_{\pi}(s,a)$.
2. A recursive formulation.
3. A successive approximation using: $q_0, q_1, q_2, \dots$

Explain how these equations can be used for **iterative policy evaluation of action values**.

Exercise 4.4 — Non-Terminating Policies

In some undiscounted episodic tasks, certain policies may **never terminate**.

For example, an agent in Gridworld could move back and forth between two states forever.

In such cases:

$$v_{\pi}(s) = -\infty$$

may occur, causing the iterative policy evaluation algorithm to never terminate.

Question:

As a practical matter, how could you modify the algorithm to guarantee termination?

Assumption:

Eventual termination is guaranteed under the **optimal policy**.

Exercise 4.5 — Programming: Jack's Car Rental

Implement **Policy Iteration** and solve Jack's Car Rental problem with the following modifications:

- One employee can move **one car from location 1 to location 2 for free** each night.
- Every additional car moved from location 1 to location 2 costs **\$2**.
- Moving cars from location 2 to location 1 costs **\$2 per car**.
- Each location has limited parking.
- If more than **10 cars** remain overnight at a location, an additional **\$4 parking cost** is charged.

Tasks:

1. Implement policy iteration.
2. First reproduce the results of the original Jack's Car Rental problem.
3. Add the new transportation and parking costs.

4. Compare the resulting policies.
5. If computation is too slow, reduce all car-related numbers by half.

Exercise 4.6 — Policy Iteration for Action Values

Define **Policy Iteration using action values** $q^\pi(s, a)$.

Develop a complete algorithm for computing:

$q^*(s, a)$

analogous to the policy iteration algorithm used to compute v^* .

Your algorithm should include:

1. Initialization
2. Policy evaluation
3. Policy improvement
4. Policy-stability check
5. Termination condition

Exercise 4.7 — ϵ -Soft Policies

Suppose we only consider ϵ -soft policies, where every action must have at least:

$$\frac{\epsilon}{|A(s)|}$$

probability of being selected.

Describe qualitatively how the following steps of Policy Iteration would change:

1. **Policy Improvement**
2. **Policy Evaluation**
3. **Initialization**

Focus on how the requirement of non-zero probability for every action affects the algorithm.

Exercise 4.8 — Gambler's Problem

Why does the optimal policy for the **Gambler's Problem** have such a surprising form?

In particular:

- At capital 5050, the gambler bets the entire amount.
- At capital 5151, the gambler does **not** bet the entire amount.

Question:

Explain why this behavior can be optimal.

Consider the relationship between:

- the current capital,
 - the goal of reaching \$100,
 - the probability of winning,
 - and the possible stakes.
-

Exercise 4.9 — Programming: Value Iteration

Implement **Value Iteration** for the Gambler's Problem.

Solve the problem for:

$$p_h=0.25$$

and

$$p_h=0.55$$

Use two terminal states:

$$V(0)=0, V(100)=1$$

Tasks:

1. Implement value iteration.
 2. Plot the value function after successive sweeps.
 3. Plot the final optimal policy.
 4. Compare the results for $p_h=0.25$ and $p_h=0.55$
 5. Investigate whether the results remain stable as: $\theta \rightarrow 0$
-

Exercise 4.10 — Value Iteration for Action Values

Derive the action-value version of the **Value Iteration backup**.

Starting from:

$$V_{k+1}(s) = \max_a E[R_{t+1} + \gamma V_k(S_{t+1})]$$

Derive the corresponding update rule for:

$$q_{k+1}(s, a)$$

Explain the difference between the value-iteration backup for V and the one for Q.

Exercise 11— 3×3 gridworld

Consider the following **3×3 gridworld**:

A	B	C
A	B	C
D	E	F
G	H	T

- T is the terminal state.
- Actions are:

$A = \{\text{up, down, left, right}\}$

- Each move gives reward: $R=-1$
- If an action would move the agent outside the grid, it remains in the same state.
- Discount factor: $\gamma=0.9$
- The agent follows a **deterministic policy**.
- Current values are:

$$V(A) = -5, \quad V(B) = -4, \quad V(C) = -3$$

$$V(D) = -4, \quad V(E) = -3, \quad V(F) = -2$$

$$V(G) = -3, \quad V(H) = -2, \quad V(T) = 0$$

Questions

(a) For state H, calculate the value of taking the **right** action using:

$$q_{\pi}(s, a) = r + \gamma V(s')$$

(b) For state HH, calculate the values of **up, left, right, and down**.

(c) Use:

$$\pi'(H) = \arg \max_a q_{\pi}(H, a)$$

to determine the new greedy action at H.

(d) Now perform one **value-iteration backup** for state H:

$$V_{\text{new}}(H) = \max_a [r + \gamma V(s')]$$

(e) Explain why the value-iteration equation uses **max_a** while policy evaluation uses the policy probabilities $\pi(a|s)$.